Module 5: Assembler and Disassembler Project

MIPS FizzBuzz to machine code (.mhc-ascii and .mhc-bin) and back

Student	Jesus Resendiz
Course	CS240
Date	October 26, 2026
Status	Assembler/disassembler verified; manual MARS screenshot still to be added

Contents

1. Parts 1 and 2: hand assembly and binary bytes
2. README and design decisions
3. Original FizzBuzz source
4. Assembler source code
5. Disassembler source code
6. Machine-code output files
7. Reconstructed assembly source
8. Test evidence
9. Sources and collaboration

1. Parts 1 and 2

Part 1: Assemble instructions by hand

Register numbers: $zero = 0, $t0 = 8, $t1 = 9, $t2 = 10, and $t3 = 11. I-type layout: opcode (6) | rs (5) | rt (5) | immediate (16). R-type layout: opcode (6) | rs (5) | rt (5) | rd (5) | shamt (5) | funct (6).

Instruction	Format	Fields	32-bit binary	Hex
`addi $t0, $zero, 3`	I-type	`001000 00000 01000 0000000000000011`	`00100000000010000000000000000011`	`0x20080003`
`addi $t1, $zero, 5`	I-type	`001000 00000 01001 0000000000000101`	`00100000000010010000000000000101`	`0x20090005`
`add $t2, $t0, $t1`	R-type	`000000 01000 01001 01010 00000 100000`	`00000001000010010101000000100000`	`0x01095020`
`sub $t3, $t1, $t0`	R-type	`000000 01001 01000 01011 00000 100010`	`00000001001010000101100000100010`	`0x01285822`

Part 2: binary bytes

Each 32-bit instruction is divided into four 8-bit bytes. The bytes are shown most-significant first (big-endian).

Instruction	8-bit bytes	Hex bytes
`addi $t0, $zero, 3`	`00100000 | 00001000 | 00000000 | 00000011`	`20 | 08 | 00 | 03`
`addi $t1, $zero, 5`	`00100000 | 00001001 | 00000000 | 00000101`	`20 | 09 | 00 | 05`
`add $t2, $t0, $t1`	`00000001 | 00001001 | 01010000 | 00100000`	`01 | 09 | 50 | 20`
`sub $t3, $t1, $t0`	`00000001 | 00101000 | 01011000 | 00100010`	`01 | 28 | 58 | 22`

The mhc-ascii format is a text file containing readable 0 and 1 characters. A 32-bit instruction requires 32 text characters, excluding spaces and line breaks. The mhc-bin format contains the actual machine-code bytes, so a 32-bit instruction requires four bytes. The ASCII form is easier for humans to read, while the binary form is compact and stores the raw byte values.

2. README and design decisions

````
# Module 5: MIPS Assembler and Disassembler (FizzBuzz)

Two Java programs that turn the course FizzBuzz program into machine code and back.

- `Assembler.java` reads `fizzbuzz.asm` and writes `fizzbuzz.mhc-ascii` and `fizzbuzz.mhc-bin`.
- `Disassembler.java` reads either of those files and writes a MIPS assembly file.

## Requirements

- Java 8 or newer with the compiler (`javac`, part of the JDK).
- Programs read and write real files, so run them from a terminal. A browser runtime such as Codevium cannot write files.
- Python 3 is needed only for the optional test emulator `tests/mhc_run.py` and for `run_tests.sh`.

## Files

```
fizzbuzz.asm                   original FizzBuzz source (input)
Assembler.java                 Part 3: assembler
Disassembler.java              Part 4: disassembler
fizzbuzz.mhc-ascii             assembler output, readable 0 and 1 characters
fizzbuzz.mhc-bin               assembler output, raw bytes
fizzbuzz.reconstructed.asm     disassembler output
run_tests.sh, tests/           test script, test inputs, test emulator
test_evidence.txt              output of run_tests.sh
```

## How to run

```
javac Assembler.java Disassembler.java

java Assembler fizzbuzz.asm
java Disassembler fizzbuzz.mhc-bin   fizzbuzz.reconstructed.asm
java Disassembler fizzbuzz.mhc-ascii other.asm

java Assembler fizzbuzz.reconstructed.asm check/fizzbuzz
cmp fizzbuzz.mhc-bin check/fizzbuzz.mhc-bin
```

- `java Assembler [--listing] <input.asm> [output-base]` writes `<output-base>.mhc-ascii` and `<output-base>.mhc-bin`. The base name defaults to the
- `java Disassembler <input> [output.asm]` detects the format on its own. The output name defaults to `<input>.asm`.
- On Windows use `fc /b a b` instead of `cmp a b`.
- `bash run_tests.sh` runs every test and prints a PASS or FAIL line for each (27 checks).

## Supported features

- Instructions: add, addi, addiu, addu, sub, ori, lui, div, mfhi, sll, beq, bne, j, jal, jr, lw, sw, syscall.
- Pseudoinstructions: li, la, move, rem (nonzero immediate divisor, uses $at), nop.
- Directives: .text, .data, .asciiz (one ASCII string per directive), .globl, .global.
- Labels (code and data), `#` comments, decimal, hex (0x) and character ('A', '\n') immediates, registers by name ($t0) or number ($8).
- Not supported: every other instruction or directive. They are reported as errors.

## Addresses, byte order and timing

- Text segment starts at 0x00400000. Data segment starts at 0x10010000.
- Entry address is the label `main` (the text base if there is no `main`).
- All 32-bit words and all header fields are big-endian: the most significant byte comes first.
- No delay slots (delayed branching off), which is the MARS default.

## File layout: .mhc-bin

All numbers are 32-bit big-endian.

```
offset  size  field
0       4     magic "MHCB" (0x4D 0x48 0x43 0x42)
4       4     text base address   (0x00400000)
8       4     text size in bytes  (4 x number of instructions)
12      4     data base address   (0x10010000)
16      4     data size in bytes
20      4     entry address
24      T     text: one 4-byte word per instruction
24+T    D     data: raw bytes of the .asciiz strings, NUL terminators included
```

For FizzBuzz: 24 + 180 + 10 = 214 bytes.

## File layout: .mhc-ascii

```
MHC-ASCII 1
ORDER big-endian
ENTRY 0x00400000
TEXT 0x00400000 words=45     <- 45 lines, each exactly 32 characters of 0 and 1
DATA 0x10010000 bytes=10     <- 8-character groups, up to 4 per line
END
```
````

Telling instructions from data

Both formats carry the sizes of the two segments. In `.mhc-bin` the header gives the text size and data size, so the first T bytes after the header a

What the disassembler reconstructs

- Each NUL-terminated string in the data segment becomes `str_N: .asciiz "..."`.
- Each instruction word becomes one real instruction. A target of a branch or jump gets a label (`L_` plus its address); the entry address is named `
- A `lui` and `ori` pair that loads the address of a string is written as `la`. `sll $zero, $zero, 0` is written as `nop`.
- Comments, original label names and the pseudoinstructions `li`, `move` and `rem` are not recovered. The real instructions they expand to are writte

Errors

Invalid input stops the program with exit status 1, writes a message to standard error and creates no output files. Examples:

```
Assembly failed: bad.asm: line 3: Unsupported instruction: mul (supported: ...)  [mul $t1, $t0, $t0]
Assembly failed: bad.asm: line 3: Undefined label: Missing  [j Missing]
Disassembly failed: bad.mhc-bin: File size mismatch: the header declares 180 text bytes and 10 data bytes (214 bytes in total) but the file has 100 b
```

Sources and collaboration

- `Assembler.java` is based on the starter assembler provided by the instructor. It was extended here to read files, write the two output formats, re
- `Disassembler.java`, `run_tests.sh` and `tests/mhc_run.py` were written for this project with help from Claude (an AI assistant made by Anthropic).
- Student: [your name]. Edit this section to match your course's collaboration policy.

3. Original FizzBuzz source

```
.data
fizz: .asciiz "Fizz"
buzz: .asciiz "Buzz"

.text
.globl main

main:
    li $t1, 1
    li $t2, 101

Loop:
    jal PrintItem
    jal PrintNewLine

    addi $t1, $t1, 1
    bne $t1, $t2, Loop

    jal Exit


# Prints Fizz, Buzz, FizzBuzz, or the number
PrintItem:
    # Save the return address before calling other procedures
    addi $sp, $sp, -8
    sw $ra, 0($sp)

    rem $t3, $t1, 3
    rem $t4, $t1, 5

    # If divisible by 3, print Fizz
    bne $t3, $zero, CheckBuzz
    jal PrintFizz

CheckBuzz:
    # If divisible by 5, print Buzz
    bne $t4, $zero, CheckNumber
    jal PrintBuzz
    j PrintItemDone

CheckNumber:
    # If Fizz was printed, skip the number
    beq $t3, $zero, PrintItemDone

    jal PrintInt

PrintItemDone:
    # Restore the original return address and release the stack space
    lw $ra, 0($sp)
    addi $sp, $sp, 8
    jr $ra


PrintFizz:
    li $v0, 4
    la $a0, fizz
    syscall
    jr $ra


PrintBuzz:
    li $v0, 4
    la $a0, buzz
    syscall
    jr $ra


PrintInt:
    li $v0, 1
    move $a0, $t1
    syscall
    jr $ra


PrintNewLine:
    li $v0, 11
    li $a0, '\n'
    syscall
    jr $ra


Exit:
    li $v0, 10
    syscall
```

4. Assembler source code

```
import java.io.IOException;
import java.nio.charset.StandardCharsets;
import java.nio.file.Files;
import java.nio.file.Path;
import java.nio.file.Paths;
import java.util.ArrayList;
import java.util.HashMap;
import java.util.HashSet;
import java.util.List;
import java.util.Map;
import java.util.Set;

/*
 * Toy MIPS assembler for Module 5 (Assembler and Disassembler Project). Java 8+.
 * Based on the instructor-provided starter Assembler.java; extended to read a
 * .asm file and write fizzbuzz.mhc-ascii and fizzbuzz.mhc-bin.
 *
 * Build and run:
 *   javac Assembler.java
 *   java Assembler fizzbuzz.asm              -> fizzbuzz.mhc-ascii + fizzbuzz.mhc-bin
 *   java Assembler fizzbuzz.asm out/fb       -> out/fb.mhc-ascii + out/fb.mhc-bin
 *   java Assembler --listing fizzbuzz.asm    -> also prints an annotated listing
 *
 * Two passes: expand pseudoinstructions and record labels, then encode.
 * Fixed addresses: .text = 0x00400000, .data = 0x10010000.
 * Words and all header fields use BIG-ENDIAN byte order.
 * Branch/jump timing: no delay slots (delayed branching OFF, as in MARS default).
 *
 * Real instructions:
 *   add, addi, addiu, addu, sub, ori, lui, div, mfhi, sll,
 *   beq, bne, j, jal, jr, lw, sw, syscall
 * Pseudoinstructions: li, la, move, rem, nop
 * Directives: .text, .data, .asciiz, .globl, .global
 *
 * OUTPUT FILE LAYOUTS (both describe the same image)
 *
 * .mhc-bin (raw bytes, all numbers 32-bit big-endian):
 *   offset  size  field
 *   0       4     magic "MHCB" (0x4D 0x48 0x43 0x42)
 *   4       4     text base address   (0x00400000)
 *   8       4     text size in bytes  (4 x number of instructions)
 *   12      4     data base address   (0x10010000)
 *   16      4     data size in bytes
 *   20      4     entry address       (address of label main, else text base)
 *   24      T     text bytes: one 4-byte big-endian word per instruction
 *   24+T    D     data bytes: raw .asciiz bytes, NUL terminators included
 *
 * .mhc-ascii (text file, same information written as 0 and 1 characters):
 *   MHC-ASCII 1
 *   ORDER big-endian
 *   ENTRY 0x00400000
 *   TEXT 0x00400000 words=N      then N lines of 32 characters (one instruction each)
 *   DATA 0x10010000 bytes=M      then 8-character groups, up to 4 per line
 *   END
 * The TEXT and DATA headers are how instructions are told apart from data.
 *
 * Limitations:
 * - .asciiz accepts one ASCII string per directive.
 * - rem accepts a nonzero immediate divisor and uses reserved register $at.
 *   It omits MARS's extra runtime overflow checks.
 * - Only the instructions listed above are supported; anything else is
 *   reported with its line number and source text.
 */
public class Assembler {
    static final int TEXT_BASE = 0x00400000;
    static final int DATA_BASE = 0x10010000;
    static final boolean RAW_CHARACTER_VIEW = false;
    static final byte[] MAGIC = {'M', 'H', 'C', 'B'};
    static final int HEADER_SIZE = 24;

    private static final String[] REGISTER_NAMES = {
        "zero", "at", "v0", "v1", "a0", "a1", "a2", "a3",
        "t0", "t1", "t2", "t3", "t4", "t5", "t6", "t7",
        "s0", "s1", "s2", "s3", "s4", "s5", "s6", "s7",
        "t8", "t9", "k0", "k1", "gp", "sp", "fp", "ra"
    };

    static class Instruction {
        final int line, address;
        final String text, sourceText;
        Instruction(int line, int address, String text, String sourceText) {
            this.line = line;
            this.address = address;
            this.text = text;
            this.sourceText = sourceText;
        }
    }

    static class EncodedInstruction {
        final Instruction source;
        final int word;
        final String mips;
        final Map<String, Integer> labelLines;
        EncodedInstruction(Instruction source, int word, String mips,
                           Map<String, Integer> labelLines) {
            this.source = source;
            this.word = word;
            this.mips = mips;
            this.labelLines = labelLines;
```

```
    }
}

static class Result {
    final byte[] text, data;
    final List<EncodedInstruction> instructions;
    final int entry;
    Result(byte[] text, byte[] data, List<EncodedInstruction> instructions, int entry) {
        this.text = text;
        this.data = data;
        this.instructions = instructions;
        this.entry = entry;
    }
}

private final Map<String, Integer> labels = new HashMap<>();
private final Map<String, Integer> labelLines = new HashMap<>();
private final Set<String> textLabels = new HashSet<>();
private final List<String> globals = new ArrayList<>();
private final List<Instruction> instructions = new ArrayList<>();
private final List<Byte> data = new ArrayList<>();
private boolean inData = false;
private int pc = TEXT_BASE;

public static void main(String[] args) {
    boolean listing = false;
    List<String> paths = new ArrayList<>();
    for (String arg : args) {
        if (arg.equals("--listing")) listing = true;
        else paths.add(arg);
    }
    if (paths.isEmpty() || paths.size() > 2) {
        System.out.println("Usage: java Assembler [--listing] <input.asm> [output-base]");
        System.out.println("Writes <output-base>.mhc-ascii and <output-base>.mhc-bin");
        System.out.println("(output-base defaults to the input path without .asm).");
        System.exit(2);
    }
    String input = paths.get(0);
    String base = paths.size() == 2 ? paths.get(1)
            : (input.toLowerCase().endsWith(".asm")
                ? input.substring(0, input.length() - 4) : input);
    try {
        String source;
        try {
            source = new String(Files.readAllBytes(Paths.get(input)), StandardCharsets.UTF_8);
        } catch (IOException e) {
            throw new IllegalArgumentException("Cannot read input file (not found or not readable)");
        }
        Result result = assemble(source);
        if (listing) {
            for (EncodedInstruction encoded : result.instructions) {
                System.out.println(binaryListingLine(encoded));
            }
        }
        Path asciiPath = Paths.get(base + ".mhc-ascii");
        Path binPath = Paths.get(base + ".mhc-bin");
        try {
            if (asciiPath.getParent() != null) Files.createDirectories(asciiPath.getParent());
            Files.write(asciiPath, asciiFile(result).getBytes(StandardCharsets.US_ASCII));
            Files.write(binPath, binFile(result));
        } catch (IOException e) {
            throw new IllegalArgumentException("Cannot write output files next to '" + base + "' (" + e.getClass().getSimpleName() + ")");
        }
        System.out.println("Assembled " + input + ": " + result.instructions.size()
            + " instructions, " + result.data.length + " data bytes");
        System.out.println("Wrote " + asciiPath + " and " + binPath);
    } catch (IllegalArgumentException e) {
        System.err.println("Assembly failed: " + input + ": " + e.getMessage());
        System.exit(1);
    }
}

static void putWord(java.io.ByteArrayOutputStream out, int word) {
    for (int shift = 24; shift >= 0; shift -= 8) out.write(word >>> shift);
}

// .mhc-bin: 24-byte big-endian header, then text bytes, then data bytes.
static byte[] binFile(Result r) {
    java.io.ByteArrayOutputStream out = new java.io.ByteArrayOutputStream();
    out.write(MAGIC, 0, MAGIC.length);
    putWord(out, TEXT_BASE);
    putWord(out, r.text.length);
    putWord(out, DATA_BASE);
    putWord(out, r.data.length);
    putWord(out, r.entry);
    out.write(r.text, 0, r.text.length);
    out.write(r.data, 0, r.data.length);
    return out.toByteArray();
}

// .mhc-ascii: the same image as readable 0/1 characters.
static String asciiFile(Result r) {
    StringBuilder sb = new StringBuilder();
    appendLine(sb, "MHC-ASCII 1");
    appendLine(sb, "ORDER big-endian");
    appendLine(sb, "ENTRY 0x" + hex(r.entry));
    appendLine(sb, "TEXT 0x" + hex(TEXT_BASE) + " words=" + (r.text.length / 4));
    for (int i = 0; i < r.text.length; i += 4) {
        int word = 0;
        for (int j = 0; j < 4; j++) word = (word << 8) | (r.text[i + j] & 255);
        appendLine(sb, binary(word));
    }
    appendLine(sb, "DATA 0x" + hex(DATA_BASE) + " bytes=" + r.data.length);
    for (int i = 0; i < r.data.length; i += 4) {
```

```
        StringBuilder line = new StringBuilder();
        for (int j = 0; j < 4 && i + j < r.data.length; j++) {
            if (j > 0) line.append(' ');
            String bits = Integer.toBinaryString(r.data[i + j] & 255);
            line.append("00000000".substring(bits.length())).append(bits);
        }
        appendLine(sb, line.toString());
    }
    appendLine(sb, "END");
    return sb.toString();
}

static String renderView(Result result, int view) {
    StringBuilder output = new StringBuilder();
    if (view == 1) {
        appendLine(output, "=== 1. BINARY MACHINE CODE - 32 bits per word ===");
        appendBinaryInstructions(output, result);
        appendSection(output, result.data, ".data", DATA_BASE, 2);
    } else if (view == 2) {
        appendLine(output, "\n=== 2. HEX MACHINE CODE - 8 digits per word ===");
        appendSection(output, result.text, ".text", TEXT_BASE, 16);
        appendSection(output, result.data, ".data", DATA_BASE, 16);
    } else if (view == 3) {
        appendLine(output, RAW_CHARACTER_VIEW
            ? "\n=== 3. RAW CHARACTER VIEW - control characters may disrupt output ==="
            : "\n=== 3. ASCII VIEW - control/non-ASCII bytes escaped ===");
        appendSection(output, result.text, ".text", TEXT_BASE, 0);
        appendSection(output, result.data, ".data", DATA_BASE, 0);
    } else {
        fail("View must be 1, 2, or 3.");
    }
    // println supplies the last newline; preserve the listing byte for byte.
    return output.substring(0, output.length() - 1);
}

private static void appendLine(StringBuilder output, String line) {
    output.append(line).append('\n');
}

static Result assemble(String source) {
    Assembler assembler = new Assembler();
    assembler.firstPass(source);
    if (assembler.instructions.isEmpty()) fail("No instructions found.");
    for (String name : assembler.globals) assembler.label(name);

    // Pass 2: all labels now have final addresses.
    byte[] text = new byte[assembler.instructions.size() * 4];
    List<EncodedInstruction> encoded = new ArrayList<>();
    int index = 0;
    for (Instruction instruction : assembler.instructions) {
        try {
            int word = assembler.encode(instruction);
            encoded.add(new EncodedInstruction(instruction, word,
                        assembler.resolvedMips(instruction), assembler.labelLines));
            for (int shift = 24; shift >= 0; shift -= 8) {
                text[index++] = (byte) (word >>> shift);
            }
        } catch (IllegalArgumentException e) {
            throw new IllegalArgumentException("line " + instruction.line + ": " + e.getMessage()
                + "  [" + instruction.sourceText + "]");
        }
    }
    byte[] bytes = new byte[assembler.data.size()];
    for (int i = 0; i < bytes.length; i++) bytes[i] = assembler.data.get(i);
    Integer main = assembler.labels.get("main");
    int entry = main != null && assembler.textLabels.contains("main") ? main : TEXT_BASE;
    return new Result(text, bytes, encoded, entry);
}

// Pass 1: expand pseudos BEFORE calculating following label addresses.
private void firstPass(String source) {
    if (source.startsWith("\uFEFF")) source = source.substring(1);
    String[] lines = source.split("\\r\\n|\\r|\\n");
    for (int i = 0; i < lines.length; i++) {
        try {
            String text = withoutComment(lines[i]).trim();
            while (true) {
                int colon = text.indexOf(':');
                if (colon < 0) break;
                String name = text.substring(0, colon).trim();
                if (!name.matches("[A-Za-z_][A-Za-z0-9_]*")) break;
                if (labels.containsKey(name)) fail("Duplicate label: " + name);
                labels.put(name, inData ? DATA_BASE + data.size() : pc);
                labelLines.put(name, i + 1);
                if (!inData) textLabels.add(name);
                text = text.substring(colon + 1).trim();
            }
            if (text.isEmpty()) continue;
            if (text.startsWith(".")) {
                directive(text);
            } else {
                if (inData) fail("Instructions must be in .text.");
                expand(text, i + 1);
            }
        } catch (IllegalArgumentException e) {
            throw new IllegalArgumentException("line " + (i + 1) + ": " + e.getMessage()
                + "  [" + lines[i].trim() + "]");
        }
    }
}

private void directive(String text) {
    String[] p = text.split("\\s+", 2);
    String value = p.length == 2 ? p[1].trim() : "";
```

```
    switch (p[0]) {
        case ".text":
        case ".data":
            boolean nextData = p[0].equals(".data");
            int base = nextData ? DATA_BASE : TEXT_BASE;
            if (!value.isEmpty()) {
                if (number(value) != base) fail("Only the default segment base is supported.");
                if (nextData ? !data.isEmpty() : !instructions.isEmpty()) {
                    fail("Explicit segment address must precede its contents.");
                }
            }
            inData = nextData;
            break;
        case ".globl":
        case ".global":
            if (!value.matches("[A-Za-z_][A-Za-z0-9_]*")) fail("Expected one global label.");
            globals.add(value);
            break;
        case ".asciiz":
            if (!inData) fail(".asciiz must be in .data.");
            String decoded = quoted(value, '"');
            for (int i = 0; i < decoded.length(); i++) data.add((byte) decoded.charAt(i));
            data.add((byte) 0);
            break;
        default:
            fail("Unsupported directive: " + p[0]);
    }
}

private void emit(String text, int line, String sourceText) {
    instructions.add(new Instruction(line, pc, text, sourceText));
    pc += 4;
}

private void loadImmediate(String register, int value, int line, String sourceText) {
    reg(register);
    if (value >= -32768 && value <= 32767) {
        emit("addiu " + register + ", $zero, " + value, line, sourceText);
    } else if (value >= 0 && value <= 65535) {
        emit("ori " + register + ", $zero, " + value, line, sourceText);
    } else {
        emit("lui " + register + ", " + (value >>> 16), line, sourceText);
        emit("ori " + register + ", " + register + ", " + (value & 65535), line, sourceText);
    }
}

private void expand(String text, int line) {
    String[] p = text.split("\\s+", 2);
    String op = lowerAscii(p[0]);
    String[] a = operands(p);
    switch (op) {
        case "li":
            count(a, 2);
            loadImmediate(a[0], number(a[1]), line, text);
            break;
        case "la":
            count(a, 2);
            reg(a[0]);
            if (!a[1].matches("[A-Za-z_][A-Za-z0-9_]*")) fail("la expects a label.");
            emit("lui " + a[0] + ", hi(" + a[1] + ")", line, text);
            emit("ori " + a[0] + ", " + a[0] + ", lo(" + a[1] + ")", line, text);
            break;
        case "move":
            count(a, 2);
            emit("addu " + a[0] + ", " + a[1] + ", $zero", line, text);
            break;
        case "rem":
            count(a, 3);
            if (reg(a[0]) == 1 || reg(a[1]) == 1) fail("rem reserves $at.");
            int divisor = number(a[2]);
            if (divisor == 0) fail("rem divisor cannot be zero.");
            loadImmediate("$at", divisor, line, text);
            emit("div " + a[1] + ", $at", line, text);
            emit("mfhi " + a[0], line, text);
            break;
        case "nop":
            count(a, 0);
            emit("sll $zero, $zero, 0", line, text);
            break;
        default:
            emit(text, line, text);
    }
}

private int encode(Instruction instruction) {
    String[] p = instruction.text.split("\\s+", 2);
    String op = lowerAscii(p[0]);
    String[] a = operands(p);
    switch (op) {
        case "addi":
        case "addiu":
        case "ori":
            count(a, 3);
            int immediate = value(a[2]);
            range(immediate, op.equals("ori") ? 0 : -32768,
                             op.equals("ori") ? 65535 : 32767);
            return iType(op.equals("addi") ? 8 : op.equals("addiu") ? 9 : 13,
                         reg(a[1]), reg(a[0]), immediate);
        case "lui":
            count(a, 2);
            return iType(15, 0, reg(a[0]), range(value(a[1]), 0, 65535));
        case "add":
        case "addu":
        case "sub":
```

```
            count(a, 3);
            return rType(reg(a[1]), reg(a[2]), reg(a[0]), 0,
                         op.equals("add") ? 0x20 : op.equals("addu") ? 0x21 : 0x22);
        case "div":
            count(a, 2);
            return rType(reg(a[0]), reg(a[1]), 0, 0, 0x1A);
        case "mfhi":
            count(a, 1);
            return rType(0, 0, reg(a[0]), 0, 0x10);
        case "sll":
            count(a, 3);
            return rType(0, reg(a[1]), reg(a[0]), range(number(a[2]), 0, 31), 0);
        case "beq":
        case "bne":
            count(a, 3);
            int offset = (codeLabel(a[2]) - (instruction.address + 4)) / 4;
            return iType(op.equals("beq") ? 4 : 5, reg(a[0]), reg(a[1]),
                         range(offset, -32768, 32767));
        case "j":
        case "jal":
            count(a, 1);
            int target = codeLabel(a[0]);
            if ((target & 0xF0000000) != ((instruction.address + 4) & 0xF0000000)) {
                fail("Jump target is out of range.");
            }
            return ((op.equals("j") ? 2 : 3) << 26) | ((target >>> 2) & 0x03FFFFFF);
        case "jr":
            count(a, 1);
            return rType(reg(a[0]), 0, 0, 0, 8);
        case "lw":
        case "sw":
            count(a, 2);
            String memory = a[1].replaceAll("\\s+", "");
            if (!memory.matches("[^()]+\\(\\$[A-Za-z0-9]+\\)")) {
                fail("Expected offset($register): " + a[1]);
            }
            int left = memory.indexOf('(');
            int base = reg(memory.substring(left + 1, memory.length() - 1));
            int displacement = range(number(memory.substring(0, left)), -32768, 32767);
            return iType(op.equals("lw") ? 35 : 43, base, reg(a[0]), displacement);
        case "syscall":
            count(a, 0);
            return 0x0000000C;
        default:
            throw new IllegalArgumentException("Unsupported instruction: " + op
                + " (supported: add, addi, addiu, addu, sub, ori, lui, div, mfhi, sll, beq, bne,"
                + " j, jal, jr, lw, sw, syscall, and pseudo li, la, move, rem, nop)");
    }
}

// R: opcode(6), rs(5), rt(5), rd(5), shamt(5), funct(6).
private static int rType(int rs, int rt, int rd, int shift, int function) {
    return (rs << 21) | (rt << 16) | (rd << 11) | (shift << 6) | function;
}

// I: opcode(6), rs(5), rt(5), immediate(16).
private static int iType(int opcode, int rs, int rt, int immediate) {
    return (opcode << 26) | (rs << 21) | (rt << 16) | (immediate & 0xFFFF);
}

private int label(String name) {
    Integer result = labels.get(name);
    if (result == null) fail("Undefined label: " + name);
    return result;
}

private int codeLabel(String name) {
    int address = label(name);
    if (!textLabels.contains(name)) fail("Branch/jump target is not in .text: " + name);
    return address;
}

private int value(String text) {
    if (text.startsWith("hi(") && text.endsWith(")")) {
        return label(text.substring(3, text.length() - 1)) >>> 16;
    }
    if (text.startsWith("lo(") && text.endsWith(")")) {
        return label(text.substring(3, text.length() - 1)) & 65535;
    }
    return number(text);
}

private static int reg(String name) {
    name = lowerAscii(name.trim());
    if (name.equals("$s8")) return 30;
    for (int i = 0; i < REGISTER_NAMES.length; i++) {
        if (name.equals("$" + REGISTER_NAMES[i]) || name.equals("$" + i)) return i;
    }
    throw new IllegalArgumentException("Unknown register: " + name);
}

private static int number(String text) {
    text = text.trim();
    if (text.startsWith("'")) {
        String character = quoted(text, '\'');
        if (character.length() != 1) fail("Expected one character: " + text);
        return character.charAt(0);
    }
    try {
        boolean negative = text.startsWith("-");
        String digits = text;
        if (digits.startsWith("-") || digits.startsWith("+")) digits = digits.substring(1);
        int radix = 10;
        if (digits.startsWith("0x") || digits.startsWith("0X")) {
```

```
            radix = 16;
            digits = digits.substring(2);
        } else if (digits.startsWith("0b") || digits.startsWith("0B")) {
            radix = 2;
            digits = digits.substring(2);
        }
        if (digits.isEmpty() || digits.startsWith("-") || digits.startsWith("+")) {
            fail("Invalid integer: " + text);
        }
        long result = Long.parseLong(digits, radix);
        if (negative) result = -result;
        if (result < Integer.MIN_VALUE || result > 0xFFFFFFFFL) fail("Integer too large: " + text);
        return (int) result;
    } catch (NumberFormatException e) {
        throw new IllegalArgumentException("Invalid integer: " + text);
    }
}

// Keep # inside quoted strings or character literals.
private static String withoutComment(String text) {
    char quote = 0;
    for (int i = 0; i < text.length(); i++) {
        char c = text.charAt(i);
        if (quote != 0) {
            if (c == '\\') i++;
            else if (c == quote) quote = 0;
        } else if (c == '"' || c == '\'') {
            quote = c;
        } else if (c == '#') {
            return text.substring(0, i);
        }
    }
    if (quote != 0) fail("Unclosed quote.");
    return text;
}

private static String quoted(String text, char quote) {
    if (text.length() < 2 || text.charAt(0) != quote
            || text.charAt(text.length() - 1) != quote) fail("Expected quoted literal: " + text);
    StringBuilder decoded = new StringBuilder();
    for (int i = 1; i < text.length() - 1; i++) {
        char c = text.charAt(i);
        if (c == quote) fail("Unexpected quote: " + text);
        if (c == '\\') {
            if (++i >= text.length() - 1) fail("Incomplete escape.");
            switch (text.charAt(i)) {
                case 'n': c = '\n'; break;
                case 'r': c = '\r'; break;
                case 't': c = '\t'; break;
                case '0': c = 0; break;
                case '\\': c = '\\'; break;
                case '\'': c = '\''; break;
                case '"': c = '"'; break;
                default: fail("Unsupported escape: \\" + text.charAt(i));
            }
        }
        if (c > 127) fail("This toy assembler accepts ASCII literals only.");
        decoded.append(c);
    }
    return decoded.toString();
}

private static String[] operands(String[] p) {
    if (p.length == 1 || p[1].trim().isEmpty()) return new String[0];
    // Split commas outside quotes so li $a0, ',' also works.
    List<String> result = new ArrayList<>();
    char quote = 0;
    int start = 0;
    for (int i = 0; i < p[1].length(); i++) {
        char c = p[1].charAt(i);
        if (quote != 0) {
            if (c == '\\') i++;
            else if (c == quote) quote = 0;
        } else if (c == '\'' || c == '"') {
            quote = c;
        } else if (c == ',') {
            result.add(p[1].substring(start, i).trim());
            start = i + 1;
        }
    }
    result.add(p[1].substring(start).trim());
    return result.toArray(new String[0]);
}

private static void count(String[] a, int expected) {
    if (a.length != expected) fail("Expected " + expected + " comma-separated operand(s).");
    for (String operand : a) if (operand.isEmpty()) fail("Empty operand.");
}

private static int range(int value, int min, int max) {
    if (value < min || value > max) fail("Value out of range: " + value);
    return value;
}

private static void fail(String message) {
    throw new IllegalArgumentException(message);
}

// Replace assembler-internal hi/lo expressions with the encoded immediates.
private String resolvedMips(Instruction instruction) {
    String[] p = instruction.text.split("\\s+", 2);
    String op = lowerAscii(p[0]);
    String[] a = operands(p);
    if (op.equals("lui")) return "lui " + a[0] + ", " + value(a[1]);
```

```
    if (op.equals("ori")) return "ori " + a[0] + ", " + a[1] + ", " + value(a[2]);
    return instruction.text;
}

static String binaryListingLine(EncodedInstruction encoded) {
    Instruction source = encoded.source;
    String instruction = displayInstructionWithLabels(source.sourceText, encoded.labelLines);
    if (!source.text.equals(source.sourceText)) {
        instruction += " => " + displayInstructionWithLabels(encoded.mips, encoded.labelLines);
    }
    return binary(encoded.word) + " # " + instruction + " # "
           + instructionType(encoded.word) + " # " + machineCodeBreakdown(encoded.word);
}

// Annotate only actual label operands, never matching register names,
// opcodes, numbers, or text inside character literals.
static String displayInstructionWithLabels(String source, Map<String, Integer> labelLines) {
    String displayed = displayInstruction(source);
    String[] p = source.split("\\s+", 2);
    String op = lowerAscii(p[0]);
    String[] a = operands(p);
    int operand = -1;
    boolean expression = false;
    if (op.equals("j") || op.equals("jal")) operand = 0;
    else if (op.equals("beq") || op.equals("bne")) operand = 2;
    else if (op.equals("la")) operand = 1;
    else if (op.equals("lui")) { operand = 1; expression = true; }
    else if (op.equals("ori") || op.equals("addi") || op.equals("addiu")) {
        operand = 2;
        expression = true;
    }
    if (operand < 0 || operand >= a.length) return displayed;
    String label = a[operand];
    if (expression) {
        if (!(label.startsWith("hi(") || label.startsWith("lo("))
                || !label.endsWith(")")) return displayed;
        label = label.substring(3, label.length() - 1);
    }
    Integer line = labelLines.get(label);
    if (line == null) return displayed;
    int end = displayed.lastIndexOf(label) + label.length();
    return displayed.substring(0, end) + " (" + line + ")" + displayed.substring(end);
}

// Each displayed field is a slice of the actual encoded word.
static String machineCodeBreakdown(int word) {
    String bits = binary(word);
    int opcode = word >>> 26;
    String fields = "opcode=" + bits.substring(0, 6);
    if (opcode == 2 || opcode == 3) {
        return fields + " target=" + bits.substring(6);
    }
    fields += " rs=" + bits.substring(6, 11) + " rt=" + bits.substring(11, 16);
    if (opcode == 0) {
        return fields + " rd=" + bits.substring(16, 21)
             + " shamt=" + bits.substring(21, 26) + " funct=" + bits.substring(26);
    }
    return fields + " immediate=" + bits.substring(16);
}

// Display-only notation: the ASM parser still expects standard offset[base]
// written with parentheses in the actual input. Quoted literals keep their
// values by using hex escapes for forbidden display punctuation.
static String displayInstruction(String source) {
    StringBuilder result = new StringBuilder();
    char quote = 0;
    for (int i = 0; i < source.length(); i++) {
        char c = source.charAt(i);
        if (quote == 0 && c == '(') {
            result.append('[');
        } else if (quote == 0 && c == ')') {
            result.append(']');
        } else if (c == '(' || c == ')' || c == '{' || c == '}') {
            result.append("\\x").append(hex(c).substring(6));
        } else {
            result.append(c);
            if (quote != 0 && c == '\\' && i + 1 < source.length()) {
                char next = source.charAt(i + 1);
                if (next == quote || next == '\\') {
                    result.append(next);
                    i++;
                }
            } else if (c == quote) {
                quote = 0;
            } else if (quote == 0 && (c == '\'' || c == '"')) {
                quote = c;
            }
        }
    }
    return result.toString();
}

private static void appendBinaryInstructions(StringBuilder output, Result result) {
    appendLine(output, ".text @ 0x" + hex(TEXT_BASE) + " - " + result.text.length + " bytes");
    for (EncodedInstruction encoded : result.instructions) {
        appendLine(output, binaryListingLine(encoded));
    }
}

private static void appendSection(StringBuilder output, byte[] bytes, String name, int base, int radix) {
    appendLine(output, name + " @ 0x" + hex(base) + " - " + bytes.length + " bytes");
    if (radix == 0) {
        appendLine(output, RAW_CHARACTER_VIEW ? characters(bytes) : printableCharacters(bytes));
        return;
```

```
        }
        if (bytes.length % 4 != 0) {
            appendLine(output, "[Final display word is zero-padded; raw data is not padded.]");
        }
        for (int i = 0; i < bytes.length; i += 4) {
            int word = 0;
            for (int j = 0; j < 4; j++) {
                word = (word << 8) | (i + j < bytes.length ? bytes[i + j] & 255 : 0);
            }
            if (radix == 2) {
                StringBuilder values = new StringBuilder(".byte ");
                for (int j = 0; j < 4 && i + j < bytes.length; j++) {
                    if (j > 0) values.append(", ");
                    values.append(bytes[i + j] & 255);
                }
                String bits = binary(word);
                appendLine(output, bits + " # " + values + " # DATA # "
                    + "byte0=" + bits.substring(0, 8) + " byte1=" + bits.substring(8, 16)
                    + " byte2=" + bits.substring(16, 24) + " byte3=" + bits.substring(24));
            } else {
                appendLine(output, hex(word));
            }
        }
    }

    // Formats for the real instructions this toy assembler supports.
    // jr, syscall and nop's sll expansion all use the R encoding.
    static String instructionType(int word) {
        int opcode = word >>> 26;
        if (opcode == 0) return "R-Type";
        if (opcode == 2 || opcode == 3) return "J-Type";
        return "I-Type";
    }

    static String binary(int word) {
        String text = Integer.toBinaryString(word);
        return "00000000000000000000000000000000".substring(text.length()) + text;
    }

    static String hex(int word) {
        String digits = "0123456789ABCDEF";
        StringBuilder text = new StringBuilder();
        for (int shift = 28; shift >= 0; shift -= 4) {
            text.append(digits.charAt((word >>> shift) & 15));
        }
        return text.toString();
    }

    static String characters(byte[] bytes) {
        StringBuilder text = new StringBuilder();
        for (int i = 0; i < bytes.length; i++) {
            text.append((char) (bytes[i] & 255));
        }
        return text.toString();
    }

    static String printableCharacters(byte[] bytes) {
        StringBuilder text = new StringBuilder();
        String digits = "0123456789ABCDEF";
        for (byte value : bytes) {
            int b = value & 255;
            if (b == '\\') {
                text.append("\\\\");
            } else if (b >= 32 && b <= 126 && b != '(' && b != ')' && b != '{' && b != '}') {
                text.append((char) b);
            } else {
                text.append("\\x");
                text.append(digits.charAt(b >>> 4));
                text.append(digits.charAt(b & 15));
            }
        }
        return text.toString();
    }

    private static String lowerAscii(String text) {
        StringBuilder result = new StringBuilder();
        for (int i = 0; i < text.length(); i++) {
            char c = text.charAt(i);
            result.append(c >= 'A' && c <= 'Z' ? (char) (c + 32) : c);
        }
        return result.toString();
    }
}
```

5. Disassembler source code

```
import java.io.IOException;
import java.nio.charset.StandardCharsets;
import java.nio.file.Files;
import java.nio.file.Paths;
import java.util.HashMap;
import java.util.Map;
import java.util.TreeMap;
import java.util.TreeSet;

/*
 * MIPS disassembler for Module 5 (Assembler and Disassembler Project). Java 8+.
 * Reads a file written by Assembler.java and writes a MIPS assembly file that
 * Assembler.java can assemble back into the same machine-code bytes.
 *
 * Build and run:
 *   javac Disassembler.java
 *   java Disassembler fizzbuzz.mhc-bin                -> fizzbuzz.mhc-bin.asm
 *   java Disassembler fizzbuzz.mhc-ascii              -> fizzbuzz.mhc-ascii.asm
 *   java Disassembler fizzbuzz.mhc-bin reconstructed.asm
 *
 * The input format is detected from the first bytes of the file:
 *   "MHCB"       -> .mhc-bin   (24-byte big-endian header, text bytes, data bytes)
 *   "MHC-ASCII"  -> .mhc-ascii (TEXT / DATA sections of 0 and 1 characters)
 * The exact layouts are documented at the top of Assembler.java and in the README.
 *
 * What is reconstructed:
 *   - .data: every NUL-terminated string becomes  str_N: .asciiz "..."
 *   - .text: one real instruction per 32-bit word (add, addi, addiu, addu, sub,
 *     ori, lui, div, mfhi, sll, beq, bne, j, jal, jr, lw, sw, syscall)
 *   - labels for every branch/jump target; the entry address is named main
 *   - a lui/ori pair that loads the address of a string is written as la
 *   - sll $zero, $zero, 0 is written as nop
 * Original comments, label names and other pseudoinstruction spellings (li, move,
 * rem) are not recovered; the real instructions they expand to are written instead.
 *
 * Errors (bad magic, truncated file, unknown opcode, ...) are reported with a
 * message that says what is wrong and where, and the program exits with status 1.
 */
public class Disassembler {
    static final int TEXT_BASE = 0x00400000;
    static final int DATA_BASE = 0x10010000;

    private static final String[] REG = {
        "zero", "at", "v0", "v1", "a0", "a1", "a2", "a3",
        "t0", "t1", "t2", "t3", "t4", "t5", "t6", "t7",
        "s0", "s1", "s2", "s3", "s4", "s5", "s6", "s7",
        "t8", "t9", "k0", "k1", "gp", "sp", "fp", "ra"
    };

    /** The decoded contents of an .mhc file, independent of which format it came from. */
    static class Image {
        int textBase, dataBase, entry;
        byte[] text, data;
    }

    static class DisassemblyException extends Exception {
        DisassemblyException(String message) { super(message); }
    }

    public static void main(String[] args) {
        if (args.length < 1 || args.length > 2) {
            System.out.println("Usage: java Disassembler <file.mhc-bin | file.mhc-ascii> [output.asm]");
            System.out.println("Output defaults to <input>.asm");
            System.exit(2);
        }
        String input = args[0];
        String output = args.length == 2 ? args[1] : input + ".asm";
        try {
            byte[] bytes;
            try {
                bytes = Files.readAllBytes(Paths.get(input));
            } catch (IOException e) {
                throw new DisassemblyException("Cannot read input file (not found or not readable)");
            }
            Image image = load(bytes);
            String asm = disassemble(image);
            try {
                Files.write(Paths.get(output), asm.getBytes(StandardCharsets.US_ASCII));
            } catch (IOException e) {
                throw new DisassemblyException("Cannot write output file '" + output + "' (" + e.getClass().getSimpleName() + ")");
            }
            System.out.println("Disassembled " + input + ": " + (image.text.length / 4)
                + " instructions, " + image.data.length + " data bytes");
            System.out.println("Wrote " + output);
        } catch (DisassemblyException e) {
            System.err.println("Disassembly failed: " + input + ": " + e.getMessage());
            System.exit(1);
        }
    }

    // ---------------------------------------------------------------- loading

    static Image load(byte[] bytes) throws DisassemblyException {
        if (bytes.length >= 4 && bytes[0] == 'M' && bytes[1] == 'H' && bytes[2] == 'C' && bytes[3] == 'B') {
            return loadBinary(bytes);
        }
        String start = new String(bytes, 0, Math.min(bytes.length, 9), StandardCharsets.ISO_8859_1);
        if (start.equals("MHC-ASCII")) {
            return loadAscii(new String(bytes, StandardCharsets.ISO_8859_1));
```

```
    }
    throw new DisassemblyException("Unrecognized file format: expected magic \"MHCB\" (.mhc-bin) "
        + "or first line \"MHC-ASCII 1\" (.mhc-ascii).");
}

private static int word(byte[] b, int offset) {
    return ((b[offset] & 255) << 24) | ((b[offset + 1] & 255) << 16)
         | ((b[offset + 2] & 255) << 8) | (b[offset + 3] & 255);
}

static Image loadBinary(byte[] b) throws DisassemblyException {
    if (b.length < 24) {
        throw new DisassemblyException("File is truncated: the .mhc-bin header needs 24 bytes but the file has "
            + b.length + ".");
    }
    Image image = new Image();
    image.textBase = word(b, 4);
    long textSize = word(b, 8) & 0xFFFFFFFFL;
    image.dataBase = word(b, 12);
    long dataSize = word(b, 16) & 0xFFFFFFFFL;
    image.entry = word(b, 20);
    if (24 + textSize + dataSize != b.length) {
        throw new DisassemblyException("File size mismatch: the header declares " + textSize + " text bytes and "
            + dataSize + " data bytes (" + (24 + textSize + dataSize) + " bytes in total) but the file has "
            + b.length + " bytes.");
    }
    image.text = java.util.Arrays.copyOfRange(b, 24, 24 + (int) textSize);
    image.data = java.util.Arrays.copyOfRange(b, 24 + (int) textSize, b.length);
    return image;
}

static Image loadAscii(String content) throws DisassemblyException {
    String[] lines = content.split("\\r\\n|\\r|\\n");
    Image image = new Image();
    image.textBase = TEXT_BASE;
    image.dataBase = DATA_BASE;
    image.entry = -1;
    java.io.ByteArrayOutputStream text = new java.io.ByteArrayOutputStream();
    java.io.ByteArrayOutputStream data = new java.io.ByteArrayOutputStream();
    String section = "";
    long expectedWords = -1, expectedBytes = -1;
    boolean sawEnd = false, sawHeader = false, sawOrder = false;
    for (int i = 0; i < lines.length; i++) {
        String line = lines[i];
        int hash = line.indexOf('#');
        if (hash >= 0) line = line.substring(0, hash);
        line = line.trim();
        if (line.isEmpty()) continue;
        int n = i + 1;
        if (line.startsWith("MHC-ASCII")) {
            if (!line.equals("MHC-ASCII 1")) {
                throw new DisassemblyException("line " + n + ": unsupported version '" + line + "' (expected \"MHC-ASCII 1\").");
            }
            sawHeader = true;
        } else if (line.startsWith("ORDER")) {
            if (!line.equals("ORDER big-endian")) {
                throw new DisassemblyException("line " + n + ": unsupported byte order '" + line + "' (expected \"ORDER big-endian\").");
            }
            sawOrder = true;
        } else if (line.startsWith("ENTRY")) {
            image.entry = (int) parseNumber(line.substring(5).trim(), n);
        } else if (line.startsWith("TEXT ") || line.startsWith("DATA ")) {
            String[] parts = line.split("\\s+");
            boolean isText = parts[0].equals("TEXT");
            String sizeKey = isText ? "words=" : "bytes=";
            if (parts.length != 3 || !parts[2].startsWith(sizeKey)) {
                throw new DisassemblyException("line " + n + ": expected \"" + parts[0] + " <address> " + sizeKey + "<count>\" but found '" + line + "'.");
            }
            long address = parseNumber(parts[1], n);
            long count = parseNumber(parts[2].substring(sizeKey.length()), n);
            if (isText) { image.textBase = (int) address; expectedWords = count; }
            else { image.dataBase = (int) address; expectedBytes = count; }
            section = parts[0];
        } else if (line.equals("END")) {
            sawEnd = true;
            break;
        } else if (section.equals("TEXT")) {
            if (!line.matches("[01]{32}")) {
                throw new DisassemblyException("line " + n + ": a TEXT line must be exactly 32 '0'/'1' characters but found '" + line + "'.");
            }
            long w = Long.parseLong(line, 2);
            for (int shift = 24; shift >= 0; shift -= 8) text.write((int) (w >>> shift));
        } else if (section.equals("DATA")) {
            for (String group : line.split("\\s+")) {
                if (!group.matches("[01]{8}")) {
                    throw new DisassemblyException("line " + n + ": a DATA group must be exactly 8 '0'/'1' characters but found '" + group + "'.");
                }
                data.write(Integer.parseInt(group, 2));
            }
        } else {
            throw new DisassemblyException("line " + n + ": unexpected content before a TEXT or DATA header: '" + line + "'.");
        }
    }
    if (!sawHeader) throw new DisassemblyException("missing \"MHC-ASCII 1\" header line.");
    if (!sawOrder) throw new DisassemblyException("missing \"ORDER big-endian\" line.");
    if (!sawEnd) throw new DisassemblyException("missing END line (file may be truncated).");
    if (expectedWords < 0) throw new DisassemblyException("missing TEXT section header.");
    if (expectedBytes < 0) throw new DisassemblyException("missing DATA section header.");
    image.text = text.toByteArray();
    image.data = data.toByteArray();
    if (image.text.length != expectedWords * 4) {
        throw new DisassemblyException("TEXT section declares " + expectedWords + " words but contains "
            + (image.text.length / 4) + ".");
    }
```

```
    if (image.data.length != expectedBytes) {
        throw new DisassemblyException("DATA section declares " + expectedBytes + " bytes but contains "
            + image.data.length + ".");
    }
    if (image.entry == -1) image.entry = image.textBase;
    return image;
}

private static long parseNumber(String s, int line) throws DisassemblyException {
    try {
        if (s.startsWith("0x") || s.startsWith("0X")) return Long.parseLong(s.substring(2), 16);
        return Long.parseLong(s);
    } catch (NumberFormatException e) {
        throw new DisassemblyException("line " + line + ": invalid number '" + s + "'.");
    }
}

// ------------------------------------------------------------ disassembling

private static String hex8(int value) {
    String h = Integer.toHexString(value);
    return "00000000".substring(h.length()) + h;
}

private static String reg(int number) {
    return "$" + REG[number];
}

static String disassemble(Image image) throws DisassemblyException {
    if (image.textBase != TEXT_BASE || image.dataBase != DATA_BASE) {
        throw new DisassemblyException("Unsupported segment base addresses (text 0x" + hex8(image.textBase)
            + ", data 0x" + hex8(image.dataBase) + "); this toolchain only supports text 0x"
            + hex8(TEXT_BASE) + " and data 0x" + hex8(DATA_BASE) + ".");
    }
    if (image.text.length % 4 != 0) {
        throw new DisassemblyException("Text size " + image.text.length + " is not a multiple of 4 bytes.");
    }
    int count = image.text.length / 4;
    int textEnd = TEXT_BASE + image.text.length;
    if (image.entry < TEXT_BASE || image.entry > textEnd || (image.entry & 3) != 0) {
        throw new DisassemblyException("Entry address 0x" + hex8(image.entry) + " is not an instruction address in the text segment.");
    }

    // Data: split into NUL-terminated strings.
    Map<Integer, String> dataLabels = new TreeMap<>();
    Map<Integer, String> dataStrings = new TreeMap<>();
    int start = 0, stringIndex = 0;
    for (int i = 0; i < image.data.length; i++) {
        int c = image.data[i] & 255;
        if (c == 0) {
            String name = "str_" + stringIndex++;
            dataLabels.put(DATA_BASE + start, name);
            dataStrings.put(DATA_BASE + start, escape(image.data, start, i, DATA_BASE + start));
            start = i + 1;
        }
    }
    if (start != image.data.length) {
        throw new DisassemblyException("Data segment does not end with a NUL byte; the last string starting at 0x"
            + hex8(DATA_BASE + start) + " is not terminated, so it cannot be written as .asciiz.");
    }

    // Pass 1: decode every word and find branch/jump targets.
    int[] words = new int[count];
    TreeSet<Integer> targets = new TreeSet<>();
    for (int i = 0; i < count; i++) {
        int w = word(image.text, i * 4);
        words[i] = w;
        int address = TEXT_BASE + i * 4;
        int op = w >>> 26;
        if (op == 4 || op == 5) {
            int target = address + 4 + ((short) w) * 4;
            checkTarget(target, textEnd, address, w);
            targets.add(target);
        } else if (op == 2 || op == 3) {
            int target = ((address + 4) & 0xF0000000) | ((w & 0x03FFFFFF) << 2);
            checkTarget(target, textEnd, address, w);
            targets.add(target);
        }
    }
    Map<Integer, String> labels = new HashMap<>();
    for (int t : targets) labels.put(t, t == image.entry ? "main" : "L_" + hex8(t));
    if (!labels.containsKey(image.entry)) labels.put(image.entry, "main");

    StringBuilder out = new StringBuilder();
    out.append("# Reconstructed by Disassembler.java\r\n");
    out.append("# Labels, formatting and pseudoinstructions are regenerated, not recovered.\r\n\r\n");
    if (!dataLabels.isEmpty()) {
        out.append(".data\r\n");
        for (Map.Entry<Integer, String> e : dataLabels.entrySet()) {
            out.append(e.getValue()).append(": .asciiz \"").append(dataStrings.get(e.getKey())).append("\"\r\n");
        }
        out.append("\r\n");
    }
    out.append(".text\r\n.globl main\r\n\r\n");

    // Pass 2: write instructions.
    for (int i = 0; i < count; i++) {
        int address = TEXT_BASE + i * 4;
        if (labels.containsKey(address)) out.append(labels.get(address)).append(":\r\n");
        int w = words[i];
        // la = lui + ori on the same register, pointing at the start of a string.
        if ((w >>> 26) == 15 && i + 1 < count && !labels.containsKey(address + 4)) {
            int next = words[i + 1];
            int rt = (w >>> 16) & 31;
```

```
                if ((w >>> 21 & 31) == 0 && (next >>> 26) == 13 && ((next >>> 21) & 31) == rt
                        && ((next >>> 16) & 31) == rt) {
                    int value = ((w & 0xFFFF) << 16) | (next & 0xFFFF);
                    if (dataLabels.containsKey(value)) {
                        out.append("    la ").append(reg(rt)).append(", ").append(dataLabels.get(value)).append("\r\n");
                        i++;
                        continue;
                    }
                }
            }
            out.append("    ").append(decode(w, address, labels)).append("\r\n");
        }
        if (labels.containsKey(textEnd)) out.append(labels.get(textEnd)).append(":\r\n");
        return out.toString();
    }

    private static void checkTarget(int target, int textEnd, int address, int w) throws DisassemblyException {
        if (target < TEXT_BASE || target > textEnd || (target & 3) != 0) {
            throw new DisassemblyException("Instruction 0x" + hex8(w) + " at address 0x" + hex8(address)
                + " branches or jumps to 0x" + hex8(target) + ", which is outside the text segment.");
        }
    }

    private static String escape(byte[] data, int from, int to, int address) throws DisassemblyException {
        StringBuilder sb = new StringBuilder();
        for (int i = from; i < to; i++) {
            int c = data[i] & 255;
            if (c == '\n') sb.append("\\n");
            else if (c == '\r') sb.append("\\r");
            else if (c == '\t') sb.append("\\t");
            else if (c == '\\') sb.append("\\\\");
            else if (c == '"') sb.append("\\\"");
            else if (c >= 32 && c <= 126) sb.append((char) c);
            else {
                throw new DisassemblyException("Data byte 0x" + Integer.toHexString(c) + " at address 0x"
                    + hex8(address + (i - from)) + " is not printable ASCII, so it cannot be written as .asciiz.");
            }
        }
        return sb.toString();
    }

    /** Decode one instruction word; every unused field must be zero so the text re-assembles to the same word. */
    static String decode(int w, int address, Map<Integer, String> labels) throws DisassemblyException {
        int op = w >>> 26;
        int rs = (w >>> 21) & 31, rt = (w >>> 16) & 31, rd = (w >>> 11) & 31, shamt = (w >>> 6) & 31;
        int funct = w & 63;
        short imm = (short) w;
        switch (op) {
            case 0:
                switch (funct) {
                    case 0x00:
                        if (rs != 0) break;
                        if (w == 0) return "nop";
                        return "sll " + reg(rd) + ", " + reg(rt) + ", " + shamt;
                    case 0x08:
                        if (rt != 0 || rd != 0 || shamt != 0) break;
                        return "jr " + reg(rs);
                    case 0x0C:
                        if (w != 0x0000000C) break;
                        return "syscall";
                    case 0x10:
                        if (rs != 0 || rt != 0 || shamt != 0) break;
                        return "mfhi " + reg(rd);
                    case 0x1A:
                        if (rd != 0 || shamt != 0) break;
                        return "div " + reg(rs) + ", " + reg(rt);
                    case 0x20: case 0x21: case 0x22:
                        if (shamt != 0) break;
                        String name = funct == 0x20 ? "add" : funct == 0x21 ? "addu" : "sub";
                        return name + " " + reg(rd) + ", " + reg(rs) + ", " + reg(rt);
                    default:
                }
                throw unsupported(w, address, "R-type instruction with funct 0b" + Integer.toBinaryString(64 | funct).substring(1)
                    + " (or non-zero unused fields)");
            case 2: case 3: {
                int target = ((address + 4) & 0xF0000000) | ((w & 0x03FFFFFF) << 2);
                return (op == 2 ? "j " : "jal ") + labels.get(target);
            }
            case 4: case 5: {
                int target = address + 4 + imm * 4;
                return (op == 4 ? "beq " : "bne ") + reg(rs) + ", " + reg(rt) + ", " + labels.get(target);
            }
            case 8: return "addi " + reg(rt) + ", " + reg(rs) + ", " + imm;
            case 9: return "addiu " + reg(rt) + ", " + reg(rs) + ", " + imm;
            case 13: return "ori " + reg(rt) + ", " + reg(rs) + ", " + (w & 0xFFFF);
            case 15:
                if (rs != 0) break;
                return "lui " + reg(rt) + ", " + (w & 0xFFFF);
            case 35: return "lw " + reg(rt) + ", " + imm + "(" + reg(rs) + ")";
            case 43: return "sw " + reg(rt) + ", " + imm + "(" + reg(rs) + ")";
            default:
        }
        throw unsupported(w, address, "opcode 0b" + Integer.toBinaryString(64 | op).substring(1));
    }

    private static DisassemblyException unsupported(int w, int address, String what) {
        return new DisassemblyException("Unsupported machine code 0x" + hex8(w) + " at address 0x" + hex8(address)
            + ": " + what + ". Supported: add, addi, addiu, addu, sub, ori, lui, div, mfhi, sll, beq, bne, j, jal, jr, lw, sw, syscall.");
    }
}
```

7. Reconstructed assembly source

```
# Reconstructed by Disassembler.java
# Labels, formatting and pseudoinstructions are regenerated, not recovered.

.data
str_0: .asciiz "Fizz"
str_1: .asciiz "Buzz"

.text
.globl main

main:
    addiu $t1, $zero, 1
    addiu $t2, $zero, 101
L_00400008:
    jal L_0040001c
    jal L_0040009c
    addi $t1, $t1, 1
    bne $t1, $t2, L_00400008
    jal L_004000ac
L_0040001c:
    addi $sp, $sp, -8
    sw $ra, 0($sp)
    addiu $at, $zero, 3
    div $t1, $at
    mfhi $t3
    addiu $at, $zero, 5
    div $t1, $at
    mfhi $t4
    bne $t3, $zero, L_00400044
    jal L_00400064
L_00400044:
    bne $t4, $zero, L_00400050
    jal L_00400078
    j L_00400058
L_00400050:
    beq $t3, $zero, L_00400058
    jal L_0040008c
L_00400058:
    lw $ra, 0($sp)
    addi $sp, $sp, 8
    jr $ra
L_00400064:
    addiu $v0, $zero, 4
    la $a0, str_0
    syscall
    jr $ra
L_00400078:
    addiu $v0, $zero, 4
    la $a0, str_1
    syscall
    jr $ra
L_0040008c:
    addiu $v0, $zero, 1
    addu $a0, $t1, $zero
    syscall
    jr $ra
L_0040009c:
    addiu $v0, $zero, 11
    addiu $a0, $zero, 10
    syscall
    jr $ra
L_004000ac:
    addiu $v0, $zero, 10
    syscall
```

8. Test evidence

```
== Build ==
PASS  javac Assembler.java Disassembler.java

== Test 1 (normal): Part 1 instructions assemble to the hand-calculated words ==
00100000000010000000000000000011 # addi $t0, $zero, 3 # I-Type
00100000000010010000000000000101 # addi $t1, $zero, 5 # I-Type
00000001000010010101000000100000 # add  $t2, $t0, $t1 # R-Type
00000001001010000101100000100010 # sub  $t3, $t1, $t0 # R-Type
Assembled tests/part1.asm: 4 instructions, 0 data bytes
Wrote out/part1.mhc-ascii and out/part1.mhc-bin
expected: ['20080003', '20090005', '01095020', '01285822']
got:      ['20080003', '20090005', '01095020', '01285822']
PASS  Part 1 encodings match

== Test 2 (normal): assemble FizzBuzz ==
00100100000010010000000000000001 # li $t1, 1 => addiu $t1, $zero, 1 # I-Type
00100100000010100000000000001100101 # li $t2, 101 => addiu $t2, $zero, 101 # I-Type
00001100000100000000000000000111 # jal PrintItem (23) # J-Type
00001100000100000000000000100111 # jal PrintNewLine (75) # J-Type
00100001001010010000000000000001 # addi $t1, $t1, 1 # I-Type
00010101001010101111111111111100 # bne $t1, $t2, Loop (12) # I-Type
00001100000100000000000000101011 # jal Exit (82) # J-Type
00100011101111011111111111111000 # addi $sp, $sp, -8 # I-Type
10101111101111110000000000000000 # sw $ra, 0[$sp] # I-Type
00100100000000010000000000000011 # rem $t3, $t1, 3 => addiu $at, $zero, 3 # I-Type
00000001001000010000000000011010 # rem $t3, $t1, 3 => div $t1, $at # R-Type
00000000000000000101100000010000 # rem $t3, $t1, 3 => mfhi $t3 # R-Type
00100100000000010000000000000101 # rem $t4, $t1, 5 => addiu $at, $zero, 5 # I-Type
00000001001000010000000000011010 # rem $t4, $t1, 5 => div $t1, $at # R-Type
00000000000000000110000000010000 # rem $t4, $t1, 5 => mfhi $t4 # R-Type
00010101011000000000000000000001 # bne $t3, $zero, CheckBuzz (35) # I-Type
00001100000100000000000000011001 # jal PrintFizz (54) # J-Type
00010101100000000000000000000010 # bne $t4, $zero, CheckNumber (41) # I-Type
00001100000100000000000000011110 # jal PrintBuzz (61) # J-Type
00001000000100000000000000010110 # j PrintItemDone (47) # J-Type
00010001011000000000000000000001 # beq $t3, $zero, PrintItemDone (47) # I-Type
00001100000100000000000000100011 # jal PrintInt (68) # J-Type
10001111101111110000000000000000 # lw $ra, 0[$sp] # I-Type
00100011101111010000000000001000 # addi $sp, $sp, 8 # I-Type
00000001111100000000000000001000 # jr $ra # R-Type
00100100000000100000000000000100 # li $v0, 4 => addiu $v0, $zero, 4 # I-Type
00111100000001000001000000000001 # la $a0, fizz (2) => lui $a0, 4097 # I-Type
00110100100001000000000000000000 # la $a0, fizz (2) => ori $a0, $a0, 0 # I-Type
00000000000000000000000000001100 # syscall # R-Type
00000011111000000000000000001000 # jr $ra # R-Type
00100100000000100000000000000100 # li $v0, 4 => addiu $v0, $zero, 4 # I-Type
00111100000001000001000000000001 # la $a0, buzz (3) => lui $a0, 4097 # I-Type
00110100100001000000000000000101 # la $a0, buzz (3) => ori $a0, $a0, 5 # I-Type
00000000000000000000000000001100 # syscall # R-Type
00000011111000000000000000001000 # jr $ra # R-Type
00100100000000100000000000000001 # li $v0, 1 => addiu $v0, $zero, 1 # I-Type
00000001001000000010000000100001 # move $a0, $t1 => addu $a0, $t1, $zero # R-Type
00000000000000000000000000001100 # syscall # R-Type
00000011111000000000000000001000 # jr $ra # R-Type
00100100000000100000000000001011 # li $v0, 11 => addiu $v0, $zero, 11 # I-Type
00100100000001000000000000001010 # li $a0, '\n' => addiu $a0, $zero, 10 # I-Type
00000000000000000000000000001100 # syscall # R-Type
00000011111000000000000000001000 # jr $ra # R-Type
00100100000000100000000000001010 # li $v0, 10 => addiu $v0, $zero, 10 # I-Type
00000000000000000000000000001100 # syscall # R-Type
Assembled fizzbuzz.asm: 45 instructions, 10 data bytes
Wrote out/fizzbuzz.mhc-ascii and out/fizzbuzz.mhc-bin
PASS  assemble fizzbuzz.asm
-rw-r--r-- 1 root root 1675 Oct 25 21:03 out/fizzbuzz.mhc-ascii
-rw-r--r-- 1 root root  214 Oct 25 21:03 out/fizzbuzz.mhc-bin
PASS  mhc-bin is 24 header + 180 text + 10 data = 214 bytes (got 214)
PASS  mhc-ascii has 45 lines of exactly 32 bit characters (got 45)

== Test 3 (normal): original machine code prints FizzBuzz (emulator) ==
PASS  mhc-bin output == FizzBuzz 1..100
PASS  mhc-ascii output == FizzBuzz 1..100

== Test 4 (normal): disassemble BOTH formats ==
Disassembled out/fizzbuzz.mhc-bin: 45 instructions, 10 data bytes
Wrote out/from_bin.asm
PASS  disassemble .mhc-bin
Disassembled out/fizzbuzz.mhc-ascii: 45 instructions, 10 data bytes
Wrote out/from_ascii.asm
PASS  disassemble .mhc-ascii
PASS  both formats give identical reconstructed source

== Test 5 (normal): reassemble reconstructed source, compare bytes ==
Assembled out/from_bin.asm: 45 instructions, 10 data bytes
Wrote out2/fizzbuzz.mhc-ascii and out2/fizzbuzz.mhc-bin
PASS  assemble reconstructed source
PASS  machine-code bytes identical (.mhc-bin)
PASS  machine-code text identical (.mhc-ascii)
PASS  reconstructed program output == FizzBuzz 1..100

== Edge cases: assembler must reject invalid input with a useful message ==
--- tests/bad_directive.asm
Assembly failed: tests/bad_directive.asm: line 2: Unsupported directive: .word  [n: .word 5]
PASS  tests/bad_directive.asm rejected (exit 1)
--- tests/bad_duplicate_label.asm
Assembly failed: tests/bad_duplicate_label.asm: line 3: Duplicate label: main  [main:]
PASS  tests/bad_duplicate_label.asm rejected (exit 1)
--- tests/bad_immediate_range.asm
Assembly failed: tests/bad_immediate_range.asm: line 2: Value out of range: 70000  [addi $t0, $zero, 70000]
PASS  tests/bad_immediate_range.asm rejected (exit 1)
```

```
--- tests/bad_register.asm
Assembly failed: tests/bad_register.asm: line 2: Unknown register: $t9x  [li $t9x, 1]
PASS  tests/bad_register.asm rejected (exit 1)
--- tests/bad_undefined_label.asm
Assembly failed: tests/bad_undefined_label.asm: line 3: Undefined label: Missing  [j Missing]
PASS  tests/bad_undefined_label.asm rejected (exit 1)
--- tests/bad_unsupported_instruction.asm
Assembly failed: tests/bad_unsupported_instruction.asm: line 3: Unsupported instruction: mul (supported: add, addi, addiu, addu, sub, ori, lui, div, mfhi, sll, beq, bne,
PASS  tests/bad_unsupported_instruction.asm rejected (exit 1)
--- missing input file
Assembly failed: tests/does_not_exist.asm: Cannot read input file (not found or not readable)
PASS  missing file rejected
PASS  no output files written for rejected input

== Edge cases: disassembler must reject invalid input with a useful message ==
--- out/bad_magic.mhc-bin
Disassembly failed: out/bad_magic.mhc-bin: Unrecognized file format: expected magic "MHCB" (.mhc-bin) or first line "MHC-ASCII 1" (.mhc-ascii).
PASS  out/bad_magic.mhc-bin rejected (exit 1)
--- out/bad_truncated.mhc-bin
Disassembly failed: out/bad_truncated.mhc-bin: File size mismatch: the header declares 180 text bytes and 10 data bytes (214 bytes in total) but the file has 100 bytes.
PASS  out/bad_truncated.mhc-bin rejected (exit 1)
--- out/bad_opcode.mhc-bin
Disassembly failed: out/bad_opcode.mhc-bin: Unsupported machine code 0x70000000 at address 0x00400000: opcode 0b011100. Supported: add, addi, addiu, addu, sub, ori, lui,
PASS  out/bad_opcode.mhc-bin rejected (exit 1)
--- out/bad_short_line.mhc-ascii
```

6. Machine-code output files

fizzbuzz.mhc-ascii

```
MHC-ASCII 1
ORDER big-endian
ENTRY 0x00400000
TEXT 0x00400000 words=45
00100100000010010000000000000001
00100100000010100000000001100101
00001100000100000000000000000111
00001100000100000000000000100111
00100001001010010000000000000001
00010101001010101111111111111100
00001100000100000000000000101011
00100011101110111111111111111000
10101111101111110000000000000000
00100100000000010000000000000011
00000001001000010000000000011010
00000000000000000101100000010000
00100100000000010000000000000101
00000001001000010000000000011010
00000000000000000110000000010000
00010101011000000000000000000001
00001100000100000000000000011001
00010101100000000000000000000010
00001100000100000000000000011110
00001000000100000000000000010110
00010001011000000000000000000001
00001100000100000000000000100011
10001111101111110000000000000000
00100011101111010000000000001000
00000011111000000000000000001000
00100100000000010000000000000100
00111100000001000001000000000001
00110100100001000000000000000000
00000000000000000000000000001100
00000011111000000000000000001000
00100100000000010000000000000100
00111100000001000001000000000001
00110100100001000000000000000101
00000000000000000000000000001100
00000011111000000000000000001000
00100100000000010000000000000001
00000001001000000010000000100001
00000000000000000000000000001100
00000011111000000000000000001000
00100100000000010000000000001011
00100100000001000000000000001010
00000000000000000000000000001100
00000011111000000000000000001000
00100100000000010000000000001010
00000000000000000000000000001100
DATA 0x10010000 bytes=10
01000110 01101001 01111010 01111010
00000000 01000010 01110101 01111010
01111010 00000000
END
```

fizzbuzz.mhc-bin hexadecimal dump

The .mhc-bin file is a raw binary file and is included separately in the project folder. The following hex dump is its readable representation. It contains a 24-byte header, 180 instruction bytes, and 10 data bytes, for 214 bytes total.

```
000000 4d 48 43 42 00 40 00 00 00 00 00 b4 10 01 00 00  >MHCB.@.........<
000010 00 00 00 0a 00 40 00 00 24 09 00 01 24 0a 00 65  >.....@..$...$..e<
000020 0c 10 00 07 0c 10 00 27 21 29 00 01 15 2a ff fc  >.......'!)...*..<
000030 0c 10 00 2b 23 bd ff f8 af bf 00 00 24 01 00 03  >...+#.......$...<
000040 01 21 00 1a 00 00 58 10 24 01 00 05 01 21 00 1a  >.!....X.$....!..<
000050 00 00 60 10 15 60 00 01 0c 10 00 19 15 80 00 02  >..`..`..........<
000060 0c 10 00 1e 08 10 00 16 11 60 00 01 0c 10 00 23  >.........`.....#<
000070 8f bf 00 00 23 bd 00 08 03 e0 00 08 24 02 00 04  >....#.......$...<
000080 3c 04 10 01 34 84 00 00 00 00 0c 03 e0 00 08     ><...4..........<
000090 24 02 00 04 3c 04 10 01 34 84 00 05 00 00 00 0c  >$...<...4.......<
0000a0 03 e0 00 08 24 02 00 01 01 20 20 21 00 00 00 0c  >....$....  !....<
0000b0 03 e0 00 08 24 02 00 0b 24 04 00 0a 00 00 00 0c  >....$...$.......<
0000c0 03 e0 00 08 24 02 00 0a 00 00 00 0c 46 69 7a 7a  >....$.......Fizz<
0000d0 00 42 75 7a 7a 00                                >.Buzz.<
0000d6
```

9. Sources and collaboration

Assembler.java is based on the starter assembler provided by the instructor and was extended to read files, write both output formats, report useful errors, and support the instructions required by the FizzBuzz program. Disassembler.java, the test runner, and emulator-based verification were completed with AI assistance from Claude and ChatGPT/Codex. The source code was compiled and the included test suite passed all 27 checks. The student should confirm that this acknowledgment matches the course collaboration policy before submitting.

Manual MARS verification remains to be added: run fizzbuzz.reconstructed.asm in MARS with delayed branching disabled, capture the FizzBuzz output, and insert the screenshot before final submission if required by the instructor.